# Informed Search

Artificial Intelligence — Computer Science — Sofia University

Bogomil Stoyanov

© 2026 Bogomil Stoyanov

# Outline

1. Heuristics

2. Greedy best-first search

3. A* search

4. Optimality of A*

5. Choosing a heuristic

6. Local search

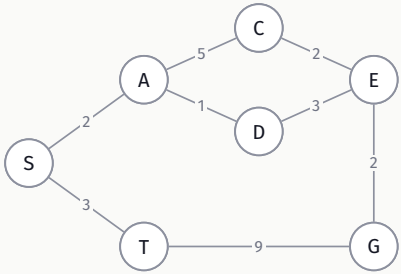
7. Summary

© 2026 Bogomil Stoyanov

# Heuristics

© 2026 Bogomil Stoyanov

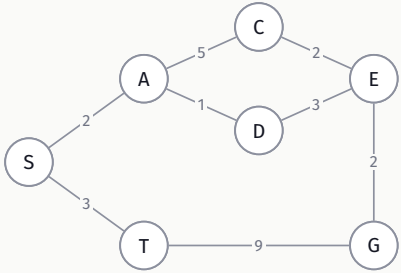
# Last week, every algorithm expanded *T*




*T* sits on the most expensive edge in the graph. Every uninformed search expanded it anyway, because none of them had any way of knowing that *G* lies over to the right.

© 2026 Bogomil Stoyanov

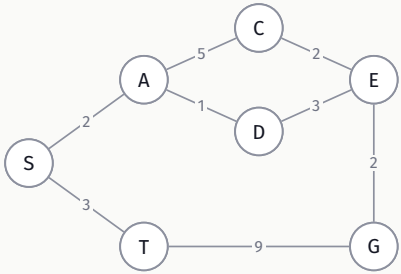
# Last week, every algorithm expanded *T*




*T* sits on the most expensive edge in the graph. Every uninformed search expanded it anyway, because none of them had any way of knowing that *G* lies over to the right.They were given the problem and nothing else.

© 2026 Bogomil Stoyanov

# Last week, every algorithm expanded *T*




*T* sits on the most expensive edge in the graph. Every uninformed search expanded it anyway, because none of them had any way of knowing that *G* lies over to the right.They were given the problem and nothing else.

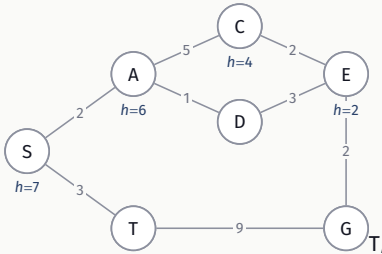
> Now we add *one number per state*: an estimate of how far it still is to the goal. one number changes the search, including, if we are careless, to make things worse.

© 2026 Bogomil Stoyanov

# The heuristic function

**Heuristic**

$h(n) \geq 0$ estimates the cost of the cheapest path from $n$ to a goal. $h(n) = 0$ for every goal state. It is an *estimate*: cheap to compute, and allowed to be wrong.



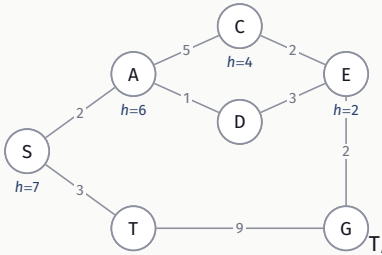

T/$h$=4/1-, D/$h$=5/1-, G/$h$=0/1-

© 2026 Bogomil Stoyanov

# The heuristic function

**Heuristic**

$h(n) \geq 0$ estimates the cost of the cheapest path from $n$ to a goal. $h(n) = 0$ for every goal state. It is an *estimate*: cheap to compute, and allowed to be wrong.




T/$h$=4/1-, D/$h$=5/1-, G/$h$=0/1- Write $h^*(n)$ for the *true* remaining cost. Here $h^* = 8, 6, 9, 4, 5, 2, 0$ for $S, A, T, C, D, E, G$, so this $h$ is right about most states and **badly optimistic about $T$** (4 against a true 9). Watch what each algorithm does with that.

© 2026 Bogomil Stoyanov

# Greedy best-first search

© 2026 Bogomil Stoyanov

# Greedy best-first: order the frontier by $h$ alone

**GREEDY = GRAPH-SEARCH, WITH:**

```
5  node ← POP-MIN( priority queue on h )
```

Uniform-cost ordered by $g$: the cost already paid. Greedy orders by $h$: the cost still to come. It ignores $g$ entirely.

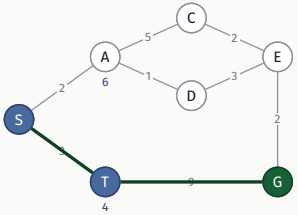
© 2026 Bogomil Stoyanov

# Greedy best-first: order the frontier by $h$ alone

**GREEDY = GRAPH-SEARCH, WITH:**

> 5 node ← POP-MIN( priority queue on $h$ )

Uniform-cost ordered by $g$: the cost already paid. Greedy orders by $h$: the cost still to come. It ignores $g$ entirely.




From $S$: $A$ has $h$=6, $T$ has $h$=4. Take $T$. From $T$: $G$ has $h$=0. Done.

**Greedy returns $S, T, G$ at cost 12.** It expanded only 3 nodes, and it walked straight into the trap.

© 2026 Bogomil Stoyanov

# Greedy best-first: the verdict

© 2026 Bogomil Stoyanov

# Greedy best-first: the verdict

**Complete?** No for tree search. It can oscillate between two states that each look good from the other. Yes for graph search on a finite space, because the explored set forbids the loop.

© 2026 Bogomil Stoyanov

# Greedy best-first: the verdict

**Complete?** No for tree search. It can oscillate between two states that each look good from the other. Yes for graph search on a finite space, because the explored set forbids the loop.

**Optimal?** No. It never looks at what a path has already cost, so it happily pays 9 to save an estimated 4.

© 2026 Bogomil Stoyanov

# Greedy best-first: the verdict

| | |
|---|---|
| **Complete?** | No for tree search. It can oscillate between two states that each look good from the other. Yes for graph search on a finite space, because the explored set forbids the loop. |
| **Optimal?** | No. It never looks at what a path has already cost, so it happily pays 9 to save an estimated 4. |
| **Time** | $O(b^m)$ worst case, but with a good $h$, close to linear in the path length. This is why people use it anyway. |

© 2026 Bogomil Stoyanov

# Greedy best-first: the verdict

| | |
|---|---|
| **Complete?** | No for tree search. It can oscillate between two states that each look good from the other. Yes for graph search on a finite space, because the explored set forbids the loop. |
| **Optimal?** | No. It never looks at what a path has already cost, so it happily pays 9 to save an estimated 4. |
| **Time** | $O(b^m)$ worst case, but with a good $h$, close to linear in the path length. This is why people use it anyway. |
| **Space** | $O(b^m)$. It keeps the whole frontier. |

© 2026 Bogomil Stoyanov

# Greedy best-first: the verdict

| | |
|---|---|
| **Complete?** | No for tree search. It can oscillate between two states that each look good from the other. Yes for graph search on a finite space, because the explored set forbids the loop. |
| **Optimal?** | No. It never looks at what a path has already cost, so it happily pays 9 to save an estimated 4. |
| **Time** | $O(b^m)$ worst case, but with a good $h$, close to linear in the path length. This is why people use it anyway. |
| **Space** | $O(b^m)$. It keeps the whole frontier. |

Greedy is the fastest thing here and the least trustworthy. Remember it when something needs to be approximately right, immediately.

© 2026 Bogomil Stoyanov

# Beam search and hill climbing

**Beam search**: greedy, but the frontier is capped at $\ell$ nodes. After each round, throw away everything but the $\ell$ best by $h$.

- space drops to $O(b\ell)$, *constant* in the depth
- and completeness goes with it: the discarded node may have been the only route to the goal

© 2026 Bogomil Stoyanov

# Beam search and hill climbing

**Beam search**: greedy, but the frontier is capped at $\ell$ nodes. After each round, throw away everything but the $\ell$ best by $h$.

- space drops to $O(b\ell)$, *constant* in the depth
- and completeness goes with it: the discarded node may have been the only route to the goal

**Hill climbing** is beam search with $\ell = 1$. Keep one state, move to its best neighbour, never look back.

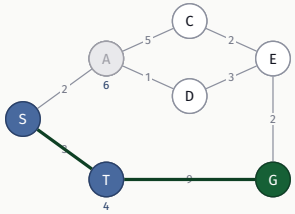
© 2026 Bogomil Stoyanov

# Beam search and hill climbing

**Beam search**: greedy, but the frontier is capped at $\ell$ nodes. After each round, throw away everything but the $\ell$ best by $h$.

- space drops to $O(b\ell)$, *constant* in the depth
- and completeness goes with it: the discarded node may have been the only route to the goal

**Hill climbing** is beam search with $\ell = 1$. Keep one state, move to its best neighbour, never look back.




On our graph, hill climbing ($\ell$=1) and beam search ($\ell$=2) both take *T* for the same reason greedy did, and both return **cost 12**. A cap on memory does not fix a bad ordering. It only makes the mistake cheaper to make.

© 2026 Bogomil Stoyanov

# A* search

© 2026 Bogomil Stoyanov

# $A^*$: count what you have paid *and* what is left

**A* = GRAPH-SEARCH, WITH:**

5 node ← POP-MIN( priority queue on $f$ )

where $f(n) = g(n) + h(n)$

9 **else if** child.state **in** frontier **with** higher $g$

replace that frontier node by child

$$\underbrace{f(n)}_{\text{est. total}} = \underbrace{g(n)}_{\text{paid}} + \underbrace{h(n)}_{\text{estimated}}$$

© 2026 Bogomil Stoyanov

# $A^*$: count what you have paid *and* what is left

**A* = GRAPH-SEARCH, WITH:**

5 node ← POP-MIN( priority queue on $f$ )

where $f(n) = g(n) + h(n)$

9 **else if** child.state **in** frontier **with** higher $g$

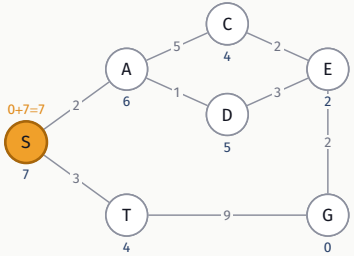
replace that frontier node by child

$$\underbrace{f(n)}_{\text{est. total}} = \underbrace{g(n)}_{\text{paid}} + \underbrace{h(n)}_{\text{estimated}}$$

Uniform-cost is $A^*$ with $h \equiv 0$. Greedy is $A^*$ with $g$ ignored. $A^*$ is the only one that uses both, and the only one of the three that is optimal.

© 2026 Bogomil Stoyanov

# $A^*$, step by step: $f = g + h$ above, $h$ below



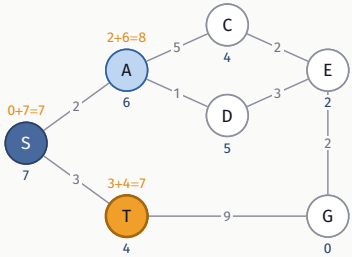

**FRONTIER: PRIORITY QUEUE ON $f$**

S(7)

$f(S) = 0 + 7 = 7$.

© 2026 Bogomil Stoyanov

# $A^*$, step by step: $f = g + h$ above, $h$ below



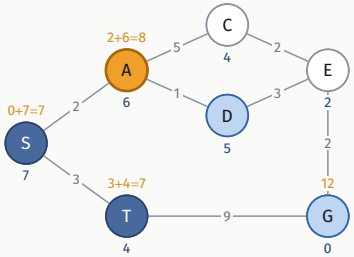

**FRONTIER: PRIORITY QUEUE ON $f$**

| T(7) | A(8) |
|---|---|

Expand *S*. *A* at $f$=8, *T* at $f$=7, so $A^*$ **takes the bait too** and expands *T* first.

© 2026 Bogomil Stoyanov

# $A^*$, step by step: $f = g + h$ above, $h$ below



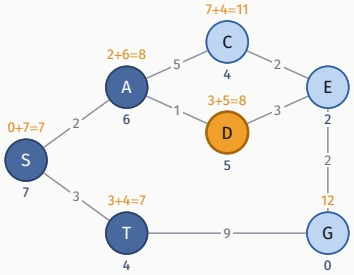

**FRONTIER: PRIORITY QUEUE ON $f$**

| A(8) | G(12) |
|---|---|

But $T$'s only child is $G$ at $g$=12, so $f(G) = 12$. That is far worse than $A^*$ expected, and $A^*$ moves on.

© 2026 Bogomil Stoyanov

# $A^*$, step by step: $f = g + h$ above, $h$ below



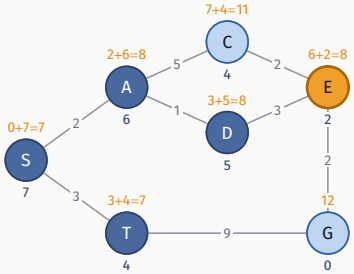

**FRONTIER: PRIORITY QUEUE ON $f$**

| D(8) | C(11) | G(12) |
|---|---|---|

Expand $A$ ($f$=8): $C$ at 11, $D$ at 8.

© 2026 Bogomil Stoyanov

# $A^*$, step by step: $f = g + h$ above, $h$ below



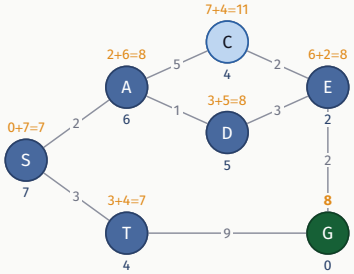

**FRONTIER: PRIORITY QUEUE ON $f$**

| E(8) | C(11) | G(12) |
|---|---|---|

Expand $D$ ($f$=8): $E$ at $f = 6 + 2 = 8$.

© 2026 Bogomil Stoyanov

# $A^*$, step by step: $f = g + h$ above, $h$ below



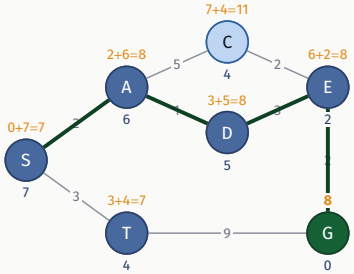

**FRONTIER: PRIORITY QUEUE ON $f$**

| G(8) | C(11) |
|---|---|

Expand $E$: $G$ at $g$=8, so $f(G) = 8 < 12$, so relax it.

© 2026 Bogomil Stoyanov

# $A^*$, step by step: $f = g + h$ above, $h$ below



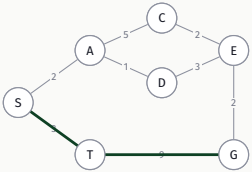

**FRONTIER: PRIORITY QUEUE ON $f$**

`C(11)`

Pop $G$ at $f$=8. $A^*$ **returns** $S, A, D, E, G$ **at cost 8: the optimum,** and it never expanded $C$.

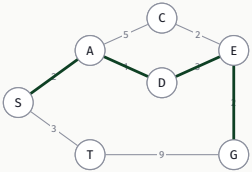
© 2026 Bogomil Stoyanov

# Greedy was fooled and stayed fooled. *A** recovered.



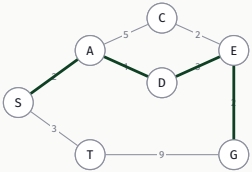

**Greedy** / beam / hill climbing
cost **12**

3 nodes expanded




**Uniform-cost**
cost **8**

7 nodes expanded

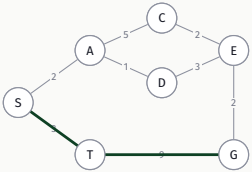



*A**
cost **8**

6 nodes expanded

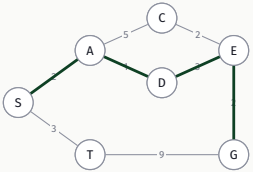
© 2026 Bogomil Stoyanov

# Greedy was fooled and stayed fooled. *A** recovered.



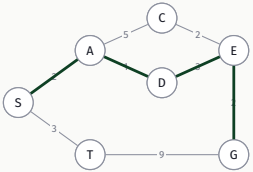

**Greedy** / beam / hill climbing
cost **12**

3 nodes expanded




**Uniform-cost**
cost **8**

7 nodes expanded




*A**
cost **8**

6 nodes expanded

*A** got uniform-cost's answer having expanded fewer nodes, on a heuristic that was actively lying about one state. The *g* term is what saved it: a misleading *h* costs *A** some wasted expansions, but it cannot change the answer, *provided h never overestimates*.

© 2026 Bogomil Stoyanov

# Optimality of A*

© 2026 Bogomil Stoyanov

# Admissibility

**Admissible**

$h$ is **admissible** if $h(n) \leq h^*(n)$ for every $n$. It never *over*estimates the remaining cost.

© 2026 Bogomil Stoyanov

# Admissibility

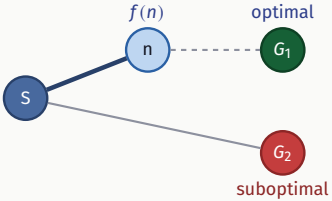
**Admissible**

$h$ is **admissible** if $h(n) \leq h^*(n)$ for every $n$. It never *over*estimates the remaining cost.

**Why it is exactly the right condition.** Suppose $A^*$ is about to pop a suboptimal goal $G_2$, while a node $n$ on the optimal path to $G_1$ is still in the frontier. Then

$$f(G_2) = g(G_2) > g(G_1) \geq f(n)$$

using $h(G_2) = 0$ and admissibility at $n$. So $f(n) < f(G_2)$ and $A^*$ would have popped $n$ first, a contradiction.




© 2026 Bogomil Stoyanov

# Admissibility

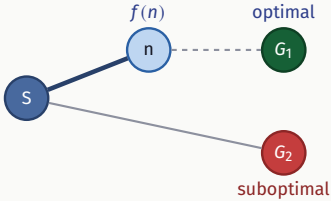
**Admissible**

$h$ is **admissible** if $h(n) \leq h^*(n)$ for every $n$. It never *over*estimates the remaining cost.

**Why it is exactly the right condition.** Suppose $A^*$ is about to pop a suboptimal goal $G_2$, while a node $n$ on the optimal path to $G_1$ is still in the frontier. Then

$$f(G_2) = g(G_2) > g(G_1) \geq f(n)$$

using $h(G_2) = 0$ and admissibility at $n$. So $f(n) < f(G_2)$ and $A^*$ would have popped $n$ first, a contradiction.



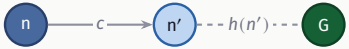

Overestimate anywhere on the optimal path and the argument breaks, and $A^*$ can then prune the very path it needed.

© 2026 Bogomil Stoyanov

# Consistency: the condition graph search needs

**Consistent (monotone)**

$h$ is **consistent** if $h(n) \le c(n, a, n') + h(n')$ for every edge. Equivalently: $f$ never *decreases* along a path.



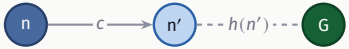

The triangle inequality: going via $n'$ cannot beat the direct estimate.

© 2026 Bogomil Stoyanov

# Consistency: the condition graph search needs

**Consistent (monotone)**

$h$ is **consistent** if $h(n) \leq c(n, a, n') + h(n')$ for every edge. Equivalently: $f$ never *decreases* along a path.




The triangle inequality: going via $n'$ cannot beat the direct estimate.Consistency makes $f$ non-decreasing,

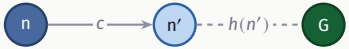
so when $A^*$ pops a node its $g$ is *already* optimal. Only then is closing it safe.

© 2026 Bogomil Stoyanov

# Consistency: the condition graph search needs

**Consistent (monotone)**

$h$ is **consistent** if $h(n) \le c(n, a, n') + h(n')$ for every edge. Equivalently: $f$ never *decreases* along a path.




The triangle inequality: going via $n'$ cannot beat the direct estimate.Consistency makes $f$ non-decreasing, so when $A^*$ pops a node its $g$ is *already* optimal. Only then is closing it safe.

With an admissible but *inconsistent* $h$, $f$ can drop along an edge, so a node may be popped before its cheapest route is known, and closing it locks the worse route in. Fixes: use a consistent $h$, or *re-open* closed nodes when a cheaper $g$ appears.

© 2026 Bogomil Stoyanov

# $A^*$: the verdict

© 2026 Bogomil Stoyanov

# $A^*$: the verdict

**Complete?** Yes, unless there are infinitely many nodes with $f \leq f(G)$.

© 2026 Bogomil Stoyanov

# $A^*$: the verdict

**Complete?** Yes, unless there are infinitely many nodes with $f \leq f(G)$.

**Optimal?** **Yes**, with an admissible $h$ for tree search, or a consistent $h$ for graph search.

© 2026 Bogomil Stoyanov

# $A^*$: the verdict

| | |
|---|---|
| **Complete?** | Yes, unless there are infinitely many nodes with $f \leq f(G)$. |
| **Optimal?** | **Yes**, with an admissible $h$ for tree search, or a consistent $h$ for graph search. |
| **Time** | Exponential in general. $A^*$ expands every node with $f < C^*$, some with $f = C^*$, and none with $f > C^*$, so the count depends on how tight $h$ is. |

© 2026 Bogomil Stoyanov

# $A^*$: the verdict

| | |
|---|---|
| **Complete?** | Yes, unless there are infinitely many nodes with $f \leq f(G)$. |
| **Optimal?** | **Yes**, with an admissible $h$ for tree search, or a consistent $h$ for graph search. |
| **Time** | Exponential in general. $A^*$ expands every node with $f < C^*$, some with $f = C^*$, and none with $f > C^*$, so the count depends on how tight $h$ is. |
| **Space** | The usual killer: every generated node stays in memory. |

© 2026 Bogomil Stoyanov

# $A^*$: the verdict

| | |
|---|---|
| **Complete?** | Yes, unless there are infinitely many nodes with $f \leq f(G)$. |
| **Optimal?** | **Yes**, with an admissible $h$ for tree search, or a consistent $h$ for graph search. |
| **Time** | Exponential in general. $A^*$ expands every node with $f < C^*$, some with $f = C^*$, and none with $f > C^*$, so the count depends on how tight $h$ is. |
| **Space** | The usual killer: every generated node stays in memory. |

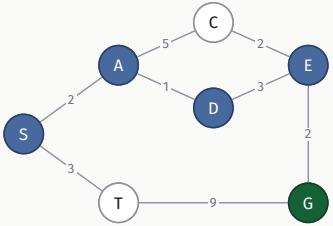
> $A^*$ is *optimally efficient*: no algorithm using the same $h$ can expand fewer nodes and still guarantee optimality. It runs out of memory long before it runs out of time.

© 2026 Bogomil Stoyanov

# Choosing a heuristic

© 2026 Bogomil Stoyanov

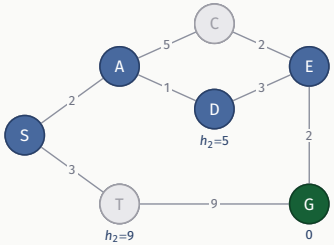
# A better heuristic beats a better algorithm




Same graph, same $A^*$. Now replace $h_1$ with $h_2 = h^*$, the exact remaining cost.

© 2026 Bogomil Stoyanov

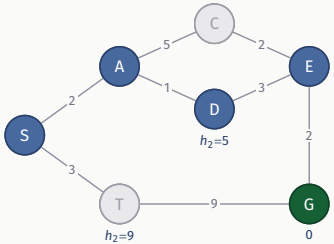
# A better heuristic beats a better algorithm




$h_2(T) = 9$ tells the truth about that edge, so $A^*$ **never expands $T$ or $C$ at all**: 5 nodes instead of 6, and it walks straight down the optimal path.

© 2026 Bogomil Stoyanov

# A better heuristic beats a better algorithm




$h_2(T) = 9$ tells the truth about that edge, so $A^*$ **never expands $T$ or $C$ at all**: 5 nodes instead of 6, and it walks straight down the optimal path.

**Dominance**

If $h_2(n) \geq h_1(n)$ for all $n$ and both are admissible, $h_2$ **dominates** $h_1$, and $A^*$ with $h_2$ never expands more nodes than with $h_1$. **Always prefer the larger admissible heuristic.**

© 2026 Bogomil Stoyanov

# Heuristics for the 8-puzzle




state




goal

$h_1$ = **number of misplaced tiles** (Hamming).
Every one of the 8 tiles is out of place, so $h_1 = 8$.

- Admissible: each misplaced tile needs at least one move.

© 2026 Bogomil Stoyanov

# Heuristics for the 8-puzzle




state




goal

$h_1$ = **number of misplaced tiles** (Hamming).
Every one of the 8 tiles is out of place, so $h_1 = 8$.

- Admissible: each misplaced tile needs at least one move.

$h_2$ = **total Manhattan distance**.
Sum $|x - x'| + |y - y'|$ over the tiles: $3+1+2+2+2+3+3+2 = 18$.

- Admissible: a tile moves one square per move, and no move shortens two tiles' distances at once.
- And $h_2 \geq h_1$ everywhere, so $h_2$ **dominates**.

© 2026 Bogomil Stoyanov

# Heuristics for the 8-puzzle




state




goal

$h_1$ = **number of misplaced tiles** (Hamming).
Every one of the 8 tiles is out of place, so $h_1 = 8$.

- Admissible: each misplaced tile needs at least one move.

$h_2$ = **total Manhattan distance**.
Sum $|x - x'| + |y - y'|$ over the tiles: 3+1+2+2+2+3+3+2 = **18**.

- Admissible: a tile moves one square per move, and no move shortens two tiles' distances at once.
- And $h_2 \geq h_1$ everywhere, so $h_2$ **dominates**.

Both come from **relaxing** the problem: drop the rule that a tile may only slide into the blank, and you get $h_2$; also let tiles stack anywhere, and you get $h_1$. *The exact cost of a relaxed problem is always an admissible heuristic for the real one.*

© 2026 Bogomil Stoyanov

# Dominance, in nodes

| **Depth** | **IDS** | $A^*$ **with** $h_1$ | $A^*$ **with** $h_2$ |
|---|---|---|---|
| 12 | 3 644 035 | 227 | 73 |
| 14 | 3 473 941 | 539 | 113 |
| 16 | — | 1 301 | 211 |
| 20 | — | 7 276 | 676 |
| 24 | $\approx 5.4 \times 10^{10}$ | 39 135 | 1 641 |

© 2026 Bogomil Stoyanov

# Dominance, in nodes

| Depth | IDS | $A^*$ with $h_1$ | $A^*$ with $h_2$ |
|---|---|---|---|
| 12 | 3 644 035 | 227 | 73 |
| 14 | 3 473 941 | 539 | 113 |
| 16 | — | 1 301 | 211 |
| 20 | — | 7 276 | 676 |
| 24 | $\approx 5.4 \times 10^{10}$ | 39 135 | 1 641 |

At depth 24, switching heuristic is a 24× saving; switching from uninformed to informed search is a $10^6\times$ saving. Effort spent on $h$ pays off far better than effort spent on the search loop, and this is the single most useful thing to remember from today.

© 2026 Bogomil Stoyanov

# $A^*$ runs out of memory. So bound it.

**IDA*(PROBLEM)**

```
1  limit ← h(root)
2  loop
3      t ← DFS-CONTOUR(root, limit)
4      if t is a solution then return t
5      if t = ∞ then return failure
6      limit ← t          ▷ smallest f that exceeded it
```

DFS-CONTOUR is a depth-first search that prunes any node with $f >$ limit and reports the smallest $f$ it pruned.

© 2026 Bogomil Stoyanov

# $A^*$ runs out of memory. So bound it.

**IDA*(PROBLEM)**

```
1 limit ← h(root)
2 loop
3     t ← DFS-CONTOUR(root, limit)
4     if t is a solution then return t
5     if t = ∞ then return failure
6     limit ← t          ▷ smallest f that exceeded it
```

DFS-CONTOUR is a depth-first search that prunes any node with $f >$ limit and reports the smallest $f$ it pruned.

**Iterative-deepening $A^*$** is last week's IDS with the limit on $f$ instead of on depth.

Same trade, same reason: $O(bd)$ space instead of exponential, at the cost of re-searching each contour from scratch.

© 2026 Bogomil Stoyanov

# $A^*$ runs out of memory. So bound it.

**IDA*(PROBLEM)**

```
1  limit ← h(root)
2  loop
3      t ← DFS-CONTOUR(root, limit)
4      if t is a solution then return t
5      if t = ∞ then return failure
6      limit ← t          ▷ smallest f that exceeded it
```

DFS-CONTOUR is a depth-first search that prunes any node with $f >$ limit and reports the smallest $f$ it pruned.

**Iterative-deepening** $A^*$ is last week's IDS with the limit on $f$ instead of on depth.

Same trade, same reason: $O(bd)$ space instead of exponential, at the cost of re-searching each contour from scratch.

> Optimal whenever $A^*$ is. In practice this is what you implement when $A^*$ dies on a 15-puzzle.

© 2026 Bogomil Stoyanov

# Local search

© 2026 Bogomil Stoyanov

# A different kind of problem

Everything so far returns a *path*. But for many problems the path is irrelevant, only the final configuration matters:

- place 8 queens so none attack
- assign 300 exams to 5 rooms and 10 slots
- choose the layout of a chip

Nobody cares *how* you arrived at a valid layout.

© 2026 Bogomil Stoyanov

# A different kind of problem

Everything so far returns a *path*. But for many problems the path is irrelevant, only the final configuration matters:

- place 8 queens so none attack
- assign 300 exams to 5 rooms and 10 slots
- choose the layout of a chip

Nobody cares *how* you arrived at a valid layout.

So drop the frontier and the explored set. Keep **one current state** and try to improve it. Memory becomes $O(1)$ in the depth, and problems with $10^{100}$ states come into range.

© 2026 Bogomil Stoyanov

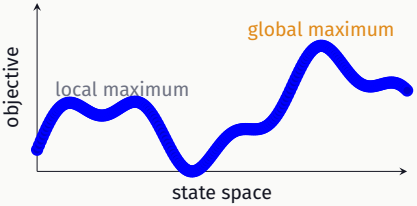
# A different kind of problem

Everything so far returns a *path*. But for many problems the path is irrelevant, only the final configuration matters:

- place 8 queens so none attack
- assign 300 exams to 5 rooms and 10 slots
- choose the layout of a chip

Nobody cares *how* you arrived at a valid layout.

So drop the frontier and the explored set. Keep **one current state** and try to improve it. Memory becomes $O(1)$ in the depth, and problems with $10^{100}$ states come into range.




© 2026 Bogomil Stoyanov

# Hill climbing, and how it gets stuck

**HILL-CLIMBING(PROBLEM)**

```
1 current ← initial state
2 loop
3     next ← best neighbour of current
4     if VALUE(next) ≤ VALUE(current)
5         return current                ▷ no way up
6     current ← next
```

“Like climbing Everest in thick fog with amnesia.”

© 2026 Bogomil Stoyanov

# Hill climbing, and how it gets stuck

**HILL-CLIMBING(PROBLEM)**

```
1 current ← initial state
2 loop
3     next ← best neighbour of current
4     if VALUE(next) ≤ VALUE(current)
5         return current                ▷ no way up
6     current ← next
```

“Like climbing Everest in thick fog with amnesia.”

- a **local maximum**: every neighbour is worse, and this is not the summit;
- a **plateau**: every neighbour is equal, so there is no gradient to follow;
- a **ridge**: every single step down, though a diagonal would go up.

© 2026 Bogomil Stoyanov

# Hill climbing, and how it gets stuck

**HILL-CLIMBING(PROBLEM)**

```
1  current ← initial state
2  loop
3      next ← best neighbour of current
4      if VALUE(next) ≤ VALUE(current)
5          return current                ▷ no way up
6      current ← next
```

"Like climbing Everest in thick fog with amnesia."

- a **local maximum**: every neighbour is worse, and this is not the summit;
- a **plateau**: every neighbour is equal, so there is no gradient to follow;
- a **ridge**: every single step down, though a diagonal would go up.

On random 8-queens, hill climbing gets stuck about **86%** of the time. **Random-restart** hill climbing: just run it again from a fresh random state, solves it nearly always, because $1/0.14 \approx 7$ restarts suffice.

© 2026 Bogomil Stoyanov

# Simulated annealing: accept bad moves, less often over time

**SIMULATED-ANNEALING(PROBLEM, SCHEDULE)**

```
1  current ← initial state
2  for t = 1, 2, ...
3      T ← schedule(t)                    ▷ cools towards 0
4      if T = 0 then return current
5      next ← a random neighbour
6      ΔE ← VALUE(next) − VALUE(current)
7      if ΔE > 0 then current ← next
8      else current ← next with probability e^(ΔE/T)
```

© 2026 Bogomil Stoyanov

# Simulated annealing: accept bad moves, less often over time

**SIMULATED-ANNEALING(PROBLEM, SCHEDULE)**

```
1  current ← initial state
2  for t = 1, 2, ...
3      T ← schedule(t)                    ▷ cools towards 0
4      if T = 0 then return current
5      next ← a random neighbour
6      ΔE ← VALUE(next) − VALUE(current)
7      if ΔE > 0 then current ← next
8      else current ← next with probability e^(ΔE/T)
```

A downhill move is accepted with probability $e^{\Delta E/T}$: likely when $T$ is high or the step is small, vanishingly unlikely once $T$ is low.

So it wanders early, escaping local maxima, and behaves like hill climbing later.

© 2026 Bogomil Stoyanov

# Simulated annealing: accept bad moves, less often over time

**SIMULATED-ANNEALING(PROBLEM, SCHEDULE)**

```
1  current ← initial state
2  for t = 1, 2, ...
3      T ← schedule(t)                    ▷ cools towards 0
4      if T = 0 then return current
5      next ← a random neighbour
6      ΔE ← VALUE(next) − VALUE(current)
7      if ΔE > 0 then current ← next
8      else current ← next with probability e^(ΔE/T)
```

A downhill move is accepted with probability $e^{\Delta E/T}$: likely when $T$ is high or the step is small, vanishingly unlikely once $T$ is low.

So it wanders early, escaping local maxima, and behaves like hill climbing later.

> If $T$ is lowered slowly enough, it finds the global optimum with probability approaching 1. "Slowly enough" is usually far too slow to wait for, so in practice you tune the schedule and accept a very good answer.

© 2026 Bogomil Stoyanov

# Summary

© 2026 Bogomil Stoyanov

# Summary of algorithms

| | **Greedy** | **Beam** | **Hill cl.** | $A^*$ | **IDA**$^*$ |
|---|---|---|---|---|---|
| Complete? | Yes[a] | No | No | Yes | Yes |
| Optimal? | No | No | No | **Yes**[b] | **Yes**[b] |
| Time | $O(b^m)$ | $O(b\ell m)$ | $O(bm)$ | exp. | exp. |
| Space | $O(b^m)$ | $\mathbf{O}(\mathbf{b}\ell)$ | $\mathbf{O}(\mathbf{b})$ | exp. | $\mathbf{O}(\mathbf{bd})$ |
| **Orders by** | $h$ | $h$, capped | $h$, one state | $g+h$ | $g+h$ |

[a] with an explored set, on a finite space

[b] admissible $h$ for tree search; consistent $h$ for graph search

© 2026 Bogomil Stoyanov

# Summary of algorithms

| | **Greedy** | **Beam** | **Hill cl.** | $A^*$ | **IDA**$^*$ |
|---|---|---|---|---|---|
| Complete? | Yes[a] | No | No | Yes | Yes |
| Optimal? | No | No | No | **Yes**[b] | **Yes**[b] |
| Time | $O(b^m)$ | $O(b\ell m)$ | $O(bm)$ | exp. | exp. |
| Space | $O(b^m)$ | $\mathbf{O(b\ell)}$ | $\mathbf{O(b)}$ | exp. | $\mathbf{O(bd)}$ |
| **Orders by** | $h$ | $h$, capped | $h$, one state | $g+h$ | $g+h$ |

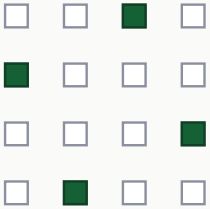
[a] with an explored set, on a finite space

[b] admissible $h$ for tree search; consistent $h$ for graph search

Read the bottom row again. Ordering by $h$ is fast and wrong; ordering by $g$ is slow and right; ordering by $g+h$ is right *and* fast, and how fast depends on how good $h$ is.

© 2026 Bogomil Stoyanov

# The missing ingredient



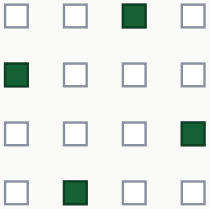
*no path, no cost, just*
*“is this assignment legal?”*

Everything in these two weeks assumed a state is a black box: we could test it, expand it, and estimate its distance, but never look *inside* it.

For a huge class of problems that is throwing information away. A state is often a set of **variables** with **values**, and the goal test is a set of **constraints** on them.

© 2026 Bogomil Stoyanov

# The missing ingredient



*no path, no cost, just “is this assignment legal?”*

Everything in these two weeks assumed a state is a black box: we could test it, expand it, and estimate its distance, but never look *inside* it.

For a huge class of problems that is throwing information away. A state is often a set of **variables** with **values**, and the goal test is a set of **constraints** on them.

> Look inside the state and you can rule out billions of assignments without ever generating them. Next session: constraint satisfaction.

© 2026 Bogomil Stoyanov